FACULTAD de CIENCIAS de la COMPUTACIÓN

Manual de Práctica 11 – Encriptación

Materia: Administración de Redes

Docente: Ana Claudia Zenteno Vázquez

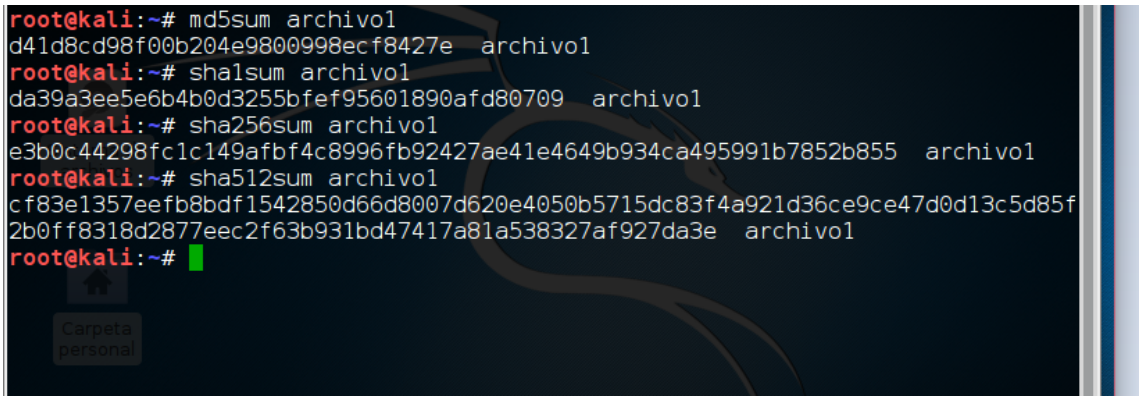
Objetivo: Que el alumno ejecute comandos como MD5 y las diferentes versiones de SHA para generar checksum de un determinado archivo.

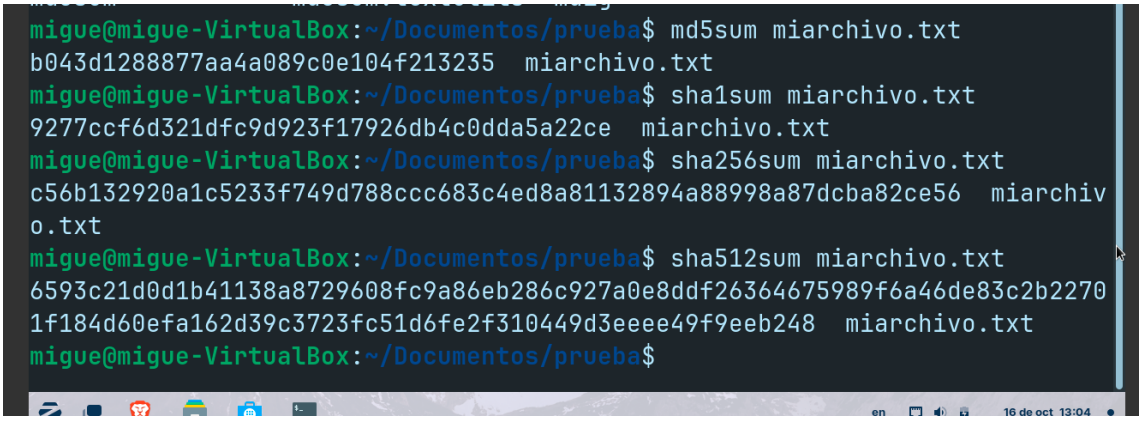
1. Realice un cuadro comparativo de descripción y opciones de los siguientes comandos:
    a. md5sum
    b. sha1sum
    c. sha256sum
    d. sha512sum

Comando	Descripción	Longitud del Hash	Opciones comunes
md5sum	Calcula un checksum utilizando el algoritmo MD5 (Message Digest 5). Genera un hash de 128 bits, representado como 32 caracteres hexadecimales. **Nota:** Se considera obsoleto para fines de seguridad, ya que es vulnerable a colisiones.	128 bits	-b (leer en modo binario) -c (comprobar hashes de un archivo) -t (leer en modo texto)
sha1sum	Calcula un checksum utilizando el algoritmo SHA-1 (Secure Hash Algorithm 1). Genera un hash de 160 bits (40 caracteres hexadecimales). **Nota:** Al igual que MD5, SHA-1 se considera inseguro y vulnerable.	160 bits	-b (leer en modo binario) -c (comprobar hashes de un archivo) -t (leer en modo texto)

sha256sum	Calcula un checksum utilizando el algoritmo SHA-256, que forma parte de la familia SHA-2. Genera un hash de 256 bits (64 caracteres hexadecimales). Actualmente se considera un estándar seguro y es ampliamente utilizado.	256 bits	-b (leer en modo binario) -c (comprobar hashes de un archivo) -t (leer en modo texto)
sha512sum	Calcula un checksum utilizando el algoritmo SHA-512, también de la familia SHA-2. Genera un hash de 512 bits (128 caracteres hexadecimales). Ofrece un nivel de seguridad aún mayor que SHA-256.	512 bits	-b (leer en modo binario) -c (comprobar hashes de un archivo) -t (leer en modo texto)

2. Seleccione un archivo aleatorio (.docx, imagen, pdf, etc) y pruebe con este los comandos listados anteriormente.
   De forma similar a





Responda las siguientes preguntas:

- ¿Por qué las salidas de cada comando son totalmente distintas en forma y dimensión?
  Las salidas son diferentes porque cada comando usa un **algoritmo matemático distinto y produce un resultado de una longitud fija y predeterminada**.

  Algoritmo: md5sum, sha1sum, y los demás son como recetas de cocina completamente diferentes. Aunque el ingrediente inicial (tu archivo) es el mismo, el proceso de "cocinarlo" es único para cada uno, por lo que el "platillo" final (el hash) es totalmente distinto.
  Dimensión: Cada algoritmo está diseñado para generar una "huella digital" de un tamaño específico:
- **MD5:** 128 bits (32 caracteres hexadecimales).
- **SHA-1:** 160 bits (40 caracteres).
- **SHA-256:** 256 bits (64 caracteres).
- **SHA-512:** 512 bits (128 caracteres).

- ¿Cuál de los comandos anteriores es mejor y por qué?

  Los mejores y más seguros son **sha256sum y sha512sum**.
  Razón principal: Son mucho más resistentes a las **"colisiones"**. Una colisión ocurre cuando dos archivos diferentes producen el mismo hash. Los algoritmos **MD5 y SHA-1 se consideran obsoletos e inseguros** porque se ha demostrado que es posible crear colisiones intencionadamente. Esto los hace poco fiables para verificar la seguridad. Para cualquier uso moderno, **SHA-256 es el estándar recomendado**.

- En sus palabras, para qué sirve un checksum.

  Un checksum es una **huella digital única de un archivo**. Sirve para **verificar la integridad de los datos**, es decir, para asegurarnos de que un archivo no ha sido modificado, alterado o corrompido. Si descargas un archivo y su checksum coincide con el publicado por el autor

3. Obtenga y comparta los códigos generados por correo electrónico. Coloque ahí mismo el archivo para que sea posible descargarlo. Pida a su compañero descargue el archivo y efectue las mismas operaciones de obtención de códigos y compárelos con el código enviado por correo. ¿es el mismo código?

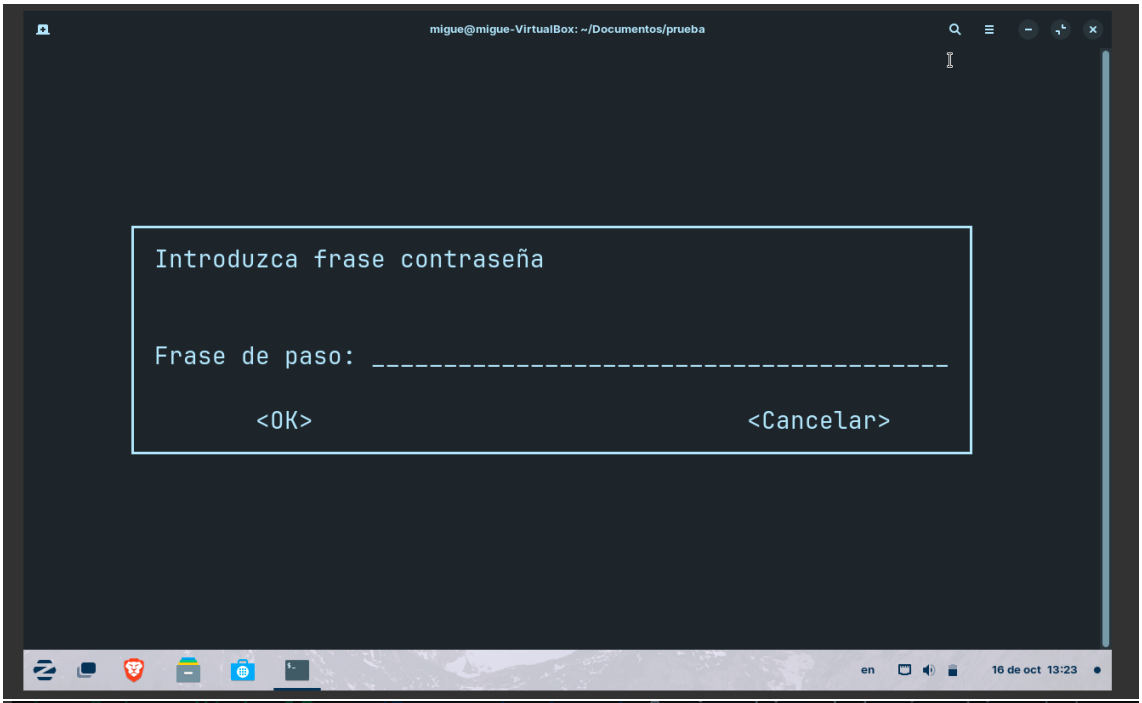
Sí. Si mi compañero descarga el archivo que le envié y no hubo ningún error en la transferencia, al ejecutar los mismos comandos (md5sum, sha256sum, etc.), debe obtener exactamente los mismos códigos que genere. Este es el principio fundamental de la verificación de integridad.

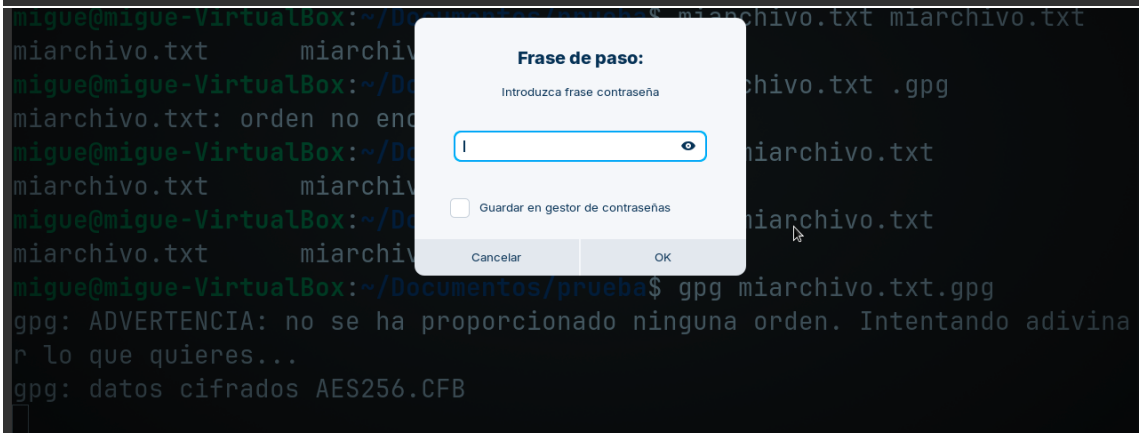
4. Si el archivo fuera modificado ¿obtiene el mismo código?
   No, en absoluto. Si mi compañero abre el archivo y cambia algo (incluso si solo agrega un espacio o borra una letra) y lo guarda, todos los checksums que genere serán completamente diferentes a los originales.
   un cambio mínimo en la entrada provoca un cambio drástico y completo en la salida del hash.

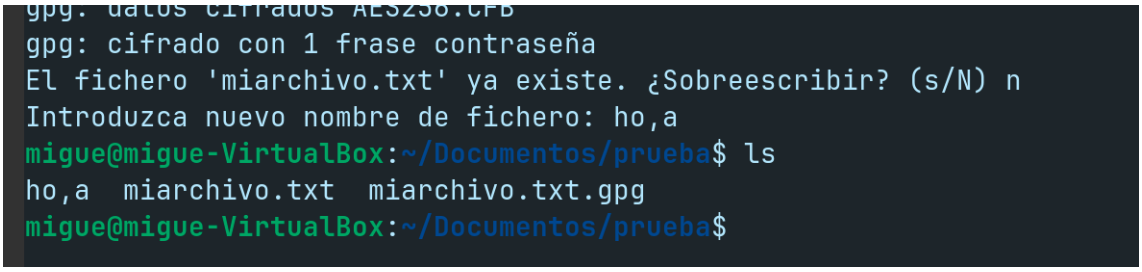
5. Refiérase al capítulo 4 del libro de Antonio Villalón que se encuentra en
   http://es.tldp.org/Manuales-LuCAS/SEGUNIX/unixsec-2.1.pdf
   y Analice los comandos crypt y pgp

Característica	**md5, sha (Hashing / Resumen)**	**crypt, pgp (Cifrado / Encriptación)**
Propósito	**Verificar Integridad:** Asegurar que un archivo no ha sido modificado.	**Garantizar Confidencialidad:** Proteger el contenido para que solo personas autorizadas puedan leerlo.
Función	Es un proceso de **una sola vía**. No puedes obtener el archivo original a partir de su hash.	Es un proceso de **dos vías**. Puedes cifrar el archivo para hacerlo ilegible y luego descifrarlo para volver al original (si tienes la clave).
Resultado	Una cadena de texto de **longitud fija** (huella digital).	Un archivo nuevo (cifrado) cuyo tamaño es **similar al original**.
Uso práctico	"Quiero saber si este archivo es el original y no ha cambiado".	"Quiero enviar este archivo de forma segura para que nadie más pueda leer su contenido".

Aplique alguno de estos dos comandos a un archivo y comparta su practica con capturas.







¿Cuál es la diferencia de usar crypt y pgp a usar md5 y sha?

La diferencia fundamental es su propósito: **MD5/SHA** se usan para **verificar la integridad** de un archivo (que no ha cambiado), mientras que **crypt/PGP** se usan para **garantizar la confidencialidad** (que nadie pueda leer su contenido).